empty `io.StringIO` object. It can accept both `Unicode Characters or 8-bit strings,` but supplying both simultaneously to the object may result in a `UnicodeError.`

```
>>> stream = StringIO()              #Initialize an empty
Object
>>> stream = StringIO('abcdef')      #Initialize using an
initial _ value
```

The purpose of `StringIO` library is to substitute file objects using strings, in an existing module that deals with files. The `StringIO` module does all of the above without actually touching the file system. Such a file like implementation of in-memory buffer is essential for `Duck Typing`.

If we wish to read the entire content of the buffer, we can use the `getvalue()` method provided with the `StringIO` class. It should be noted that `getvalue()` reads the entire content but the stream still points to the original position of the buffer, unlike the `read()` operation where the pointer is moved to the end of the file.

```
>>> stream.getvalue()
'abcdef'
>>> stream.tell()    #gives the current position of
stream pointer
0                    #points to the start of the stream
>>> stream.read()
'abcdef'
>>> stream.tell()
6                    #points to the end of the stream
```

The text buffer maintained so far is discarded whenever we call the `close()` method. There is no way to retrieve the text buffer post this method call since there is no file backing up this buffer and it was created as an in-memory file like representation of the text.

Calling a `getvalue()` method on the `StringIO` object post a `close()` call will result in a `ValueError` since the stream was already closed and discarded.

```
>>> stream.close()
>>> stream.getvalue()
```